Programmierprojekt:
Ein webbasierter Routenplaner

Felix Weitbrecht

(FMI/ALG)

http://www.fmi.uni-stuttgart.de/alg

Aufgaben und Ziele

Implementierung eines webbasierten Routenplaners

- Backend: Graphrepräsentation, Routenberechnung; läuft auf “Server”
- GUI: Web-Client (Javascript, Leaflet)
- Webserverkomponente: Verbindung zwischen Backend und GUI

Ziele / Was lernen Sie?

- Umsetzung eines nicht-trivialen Projektes in einer Kleingruppe
- Arbeiten mit nicht ganz kleinen Datenmengen

Ablauf

- Aufteilung des Projektes in 2 Phasen entsprechend den Komponenten
- Abnahme nach Ende jeder Phase
- Fortführung des Projektes nur nach erfolgreicher Abnahme

Phase I: Backend: Routenplanung

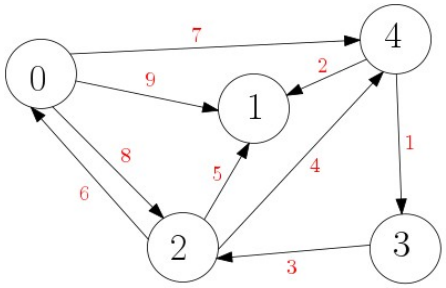
- Effiziente Darstellung von Straßennetzwerken im Speicher
- Routenplanung mittels Dijkstra
- Nächster-Knoten-Suche

Phase II: GUI/Webclient und Webserver

- Kartendarstellung in Browser
- Interaktion mit Benutzer
- Erstellung und Verknüpfung von Server und Client

Phase I: Backend – Graphdarstellung

- Auf https://fmi.uni-stuttgart.de/alg/research/stuff/ finden sich Graphen verschiedenster Größen
- 1. Aufgabe von Phase I:
  - Implementieren Sie eine Klasse, welche derartige Graphen effizient im Speicher darstellt
  - Ihre Klasse sollte auf der AdjacencyArray-Darstellung basieren um schnell ausgehende Kanten zu einem Knoten finden zu können
  - Ein Rechner mit 8GB sollte problemlos den Deutschlandgraph (n=20Mio, m=40Mio) im RAM halten können



node	offset
0	0
1	3
2	3
3	6
4	7
5	9

edge idx	src	trg	weight
0	0	1	9
1	0	2	8
2	0	4	7
3	2	0	6
4	2	1	5
5	2	4	4
6	3	2	3
7	4	1	2
8	4	3	1

Phase I: Backend – Dijkstras Algorithmus

- 2. Aufgabe von Phase I:
  - Implementieren Sie Dijkstras Algorithmus
  - Auf dem Deutschlandgraph sollte eine Anfrage wenige Sekunden (< 15s) dauern

Phase I: Backend – Nächster Knoten

- 3. Aufgabe von Phase I:
  - Geg. Eine Position Lon./Lat., finde nächsten Knoten
  - Sollte schnell genug sein, damit Nutzer nicht so lange warten müssen
    → < 1 Sekunde für Deutschlandgraph
  - 2 Möglichkeiten für Distanzberechnung:
    - Großkreisdistanzen (akkurater)
    - Euklidische Distanz der Koordinaten (einfacher)
  - Datenstruktur beliebig, d.h.: Quadtree oder Grid, etc
    - Kein Bruteforce erlaubt!

Phase II: GUI/Webclient

- 1. Aufgabe von Phase II:
  - Machen Sie sich mit der JavaScript Bibliothek Leaflet (http://www.leafletjs.com) vertraut
  - Erstellen Sie zunächst eine einfache HTML-Seite, welche:
    - Mit OpenStreetMap als Tile-Provider einen Kartenausschnitt darstellt, der Zoom- und Scrolloperationen unterstützt
    - Auf Mausclick die entsprechende Position in Längen- und Breitengrad in einem Anzeigeelement der Seite darstellt
  - Finden Sie heraus, wie man geometrische Objekte (Pfade) im GeoJSON-Format als Layer über die Karte legt

Phase II: Anbindung Webclient <->Backend

- 2. Aufgabe von Phase II:
  - Erstellen Sie einen einfachen Webserver, welcher
    - den HTML-Code bereitstellt, der von einem Webbrowser (Client) ausgeführt werden kann.
    - Anfragen (Ajax-Requests) des Clients entgegen nimmt, die entsprechenden Methoden im Backend aufruft und deren Ergebnis an den Client zurückschickt.
  - Mindestfunktionalität:
    - „Nächster Knoten“: Gegeben Längen-&Breitengrad: Rückgabe des nächsten Knotens (ID, Längen-&Breitengrad)
    - „Routenplanung“: Gegeben KnotenIDs von Start und Ziel: Berechne Route, und gebe Pfad zur Visualisierung zurück

Screenshot: Beispiel Webanwendung

Coordinates: 48.74580431750093, 9.11543369293213

Set Source: 48.744602400000005, 9.100064600000001 (134619)

Set Target: 48.745671300000005, 9.1150137 (98366)

Query

Clear

Empfehlungen

- Nutzen Sie zum Entwickeln eine IDE (Eclipse, o.Ä.) und deren Möglichkeiten wie z.B. Profiler, Debugger, Autoformatting … Trotzdem muss am Schluss die Konsole ausreichen um das Projekt zu kompilieren und auszuführen.
- Nutzen Sie ein Versionsverwaltungssystem wie z.B. git

Stolpersteine

- Encoding: US-ASCII mit Umlauten kann Probleme beim Kompilieren verursachen, besonders auf anderen Systemen
- Bei OutOfMemoryException: JVM-Parameter für Arbeitsspeicher: z.B. java -Xmx8g
- Java-Objekte benötigen viel Speicher, elementare Datentypen sind sparsamer
- Bei multidimensionalen Arrays macht es einen Unterschied ob diese als new int[3][5000000] oder new int[5000000][3] allokiert werden.
- (Hash)maps benötigen viel Speicher
- Java‘s PriorityQueue unterstützt keine decrease-key-Operationen
  - Doppeltes Einfügen bei extern gespeicherten Distanzen kann zu Fehlern in der PriorityQueue führen